

Curse of Dimensionality in the context of the **k-Nearest Neighbor (kNN)** algorithm.

In simple words:

👉 When we move from low dimensions (like a line or a 2D plane) to very high dimensions (like 100D space), distances and volumes start behaving in strange and unintuitive ways.

This makes kNN (and other distance-based algorithms) harder to use in high-dimensional data.

The Ball Example

- Imagine a **ball (or circle/sphere)** of radius 1.
- Inside it, you also take a smaller ball of radius $(1 - \epsilon)$.
 - For example: if $\epsilon = 0.1$, then the small ball radius = 0.9.

The slides are asking: **What fraction (percentage) of the volume is near the outer shell?**

Mathematically:

$$\text{Fraction outside inner ball} = 1 - (1 - \epsilon)^D$$

- D = dimension (1D, 2D, 3D, ...).
-

The Curse

Now look at what happens when **dimension increases** ($D = 2, 10, 100, \dots$):

- In low dimension (like 2D), the outer shell (grey region) is small compared to the whole ball.
- But in high dimension (like 100D), **almost all the volume is near the edges (outer shell)!**

The graph on the right shows this:

- As D increases, the curve goes up fast.
 - Meaning: for high- D , most of the volume is concentrated near the boundary of the ball.
-

Why does this matter for kNN?

- kNN relies on **distances** to find nearest neighbors.
- But in high dimensions, all points tend to be **far from each other and lie near the edges**.
- So, the concept of “nearest neighbor” becomes less meaningful → distances between nearest and farthest neighbors become almost the same.

This is one way of saying kNN suffers from the **curse of dimensionality**.

Super simple takeaway:

- In low dimensions: neighbors are close and distances make sense.
 - In high dimensions: almost everything is on the edge, distances stop being useful, and kNN struggles.
-

👉 Think of it like blowing up a balloon:

- In 2D (a circle), most area is inside.
- In 3D (a ball), more of the space is near the surface.
- In higher dimensions, **almost all space is surface**, very little is inside.

That’s the “curse of dimensionality.”

We’ll use the formula from the slide:

$$\text{Fraction outside inner ball} = 1 - (1 - \epsilon)^D$$

Let’s pick $\epsilon = 0.1$ (so inner ball radius = 0.9).

Example Calculations:

Case 1: Low dimension ($D = 2 \rightarrow$ like a circle in 2D)

$$1 - (1 - 0.1)^2 = 1 - (0.9)^2 = 1 - 0.81 = 0.19$$

👉 Only **19% of the volume** is near the outer shell.

Case 2: Medium dimension ($D = 10 \rightarrow$ like 10D space)

$$1 - (1 - 0.1)^{10} = 1 - (0.9)^{10} \approx 1 - 0.3487 = 0.6513$$

👉 **65% of the volume** is near the outer shell!

Case 3: Higher dimension ($D = 100 \rightarrow$ 100D space)

$$1 - (1 - 0.1)^{100} = 1 - (0.9)^{100} \approx 1 - 0.000026 = 0.999974$$

👉 Almost **100% of the volume** is near the outer shell!

Super Simple Takeaway:

- In **2D**, most of the volume is inside (only 19% outside).
- In **10D**, more than half (65%) is outside.
- In **100D**, almost everything (99.99%) is outside.

So, as dimensions grow, points “live on the edge” → which makes distances less meaningful for kNN.

 **Orange Example (instead of "ball")**

Think of a **round orange**.

- The **whole orange** is like all the space we have.
 - The **juicy inside** is the “middle.”
 - The **thin skin/peel** is the “edge.”
-

Case 1: Small world (2D or 3D)

- In a normal orange, the **peel is thin** compared to the inside.
 - So, most of the orange is the **juicy part** (middle).
-

Case 2: Bigger world (more dimensions)

- Now imagine if the orange was living in **10 dimensions** (weird to imagine, but let's pretend).
 - Suddenly, the **peel becomes much thicker** compared to the inside.
 - So, most of the orange is now **peel/skin**, not juice.
-

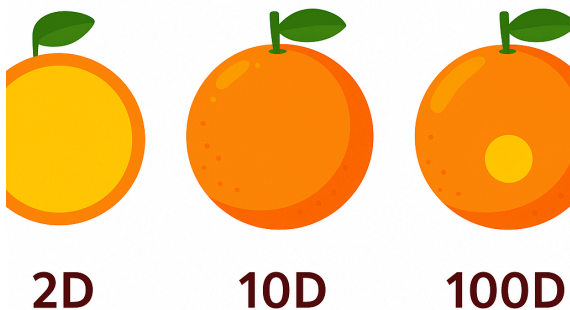
Case 3: Very high dimensions

- In **100 dimensions**, almost the **entire orange is peel**.
 - The juicy inside almost disappears.
-

Why does this matter for kNN?

- kNN tries to find **closest points (neighbors)**.
 - But if in high dimensions **almost everything is peel/edge**, then all points are far out and look similar.
 - So, kNN gets **confused** — it can't tell who is really “near” and who is “far.”
-

Orange Example



👉 Super short takeaway:

- Low dimensions = most data is in the middle.
 - High dimensions = most data is on the edge.
 - That's why kNN struggles in high dimensions.
-

Why does kNN still work in real-world data despite the Curse of Dimensionality?

Even though distances get weird in very high dimensions, **real-world data is not spread everywhere in high-dimensional space**. Instead, it usually has some structure.

1. Data lives in smaller, meaningful regions

- Even if the data is stored in 1000 dimensions, in reality it often lies in a much smaller shape (like a line, curve, or surface).
- Example:
 - A photo might have 1 million pixels → 1 million dimensions.
 - But the *important information* (like orientation of a face) can be described with just a few numbers (maybe 1 or 2 dimensions).

👉 This is why **Dimensionality Reduction** (covered later) works — we can shrink data into fewer dimensions without losing much meaning.

2. Data is smooth (neighbors are still similar)

- In real life, if two things are close, they usually behave similarly.
- Example:
 - Two emails with very similar words → likely both spam or both not spam.
 - Two face images that look alike → likely the same person.

👉 This smoothness allows kNN to still make **good predictions**.

3. Examples made simple

- **Data along a line or plane in high dimensions:**
Imagine students sitting in a classroom → although the room is 3D, their chairs are arranged on a 2D floor (a smaller region).
 - **Object orientation in an image:**
A rotating object (like a chair turning) → even though the picture has thousands of pixels, the *rotation* can be described by just **1 number (angle)**.
 - **Face recognition:**
A photo might have 50 or 70 features, but the important structure (eye distance, nose size, etc.) is much smaller and organized.
 - **Spam filter:**
An email might have thousands of possible words (dimensions), but only a few (like “free,” “offer,” “win”) actually matter.
-

✨ Super Short Takeaway:

- Real-world data is not “randomly scattered” in high dimensions.
 - It usually lies on a **smaller surface (lower dimension)** and has **smooth patterns**.
 - That’s why **kNN still works well** in practice, even if high dimensions cause problems in theory.
-